

connectivity to the MySQL server. For example:

```
connectInfo :: ConnectInfo
connectInfo = ConnectInfo { connectHost = "localhost",
                           connectPort = 3306,
                           connectUser = "user",
                           connectPassword = "password",
                           connectDatabase = "test",
                           connectOptions = [],
                           connectPath = "",
                           connectSSL = Nothing }
```

The above code to create a table can be compiled directly in GHCi and the *main* function can be executed as given below:

```
$ ghci create.hs
GHCi, version 7.6.3: http://www.haskell.org/ghc/ ? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main ( create.hs, interpreted )
Ok, modules loaded: Main.
ghci> main
Loading package array-0.4.0.1 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
Loading package old-locale-1.0.5 ... linking ... done.
Loading package time-1.4.0.1 ... linking ... done.
Loading package bytestring-0.10.0.2 ... linking ... done.
Loading package text-0.11.3.1 ... linking ... done.
Loading package blaze-builder-0.3.1.1 ... linking ... done.
Loading package primitive-0.5.0.1 ... linking ... done.
Loading package vector-0.10.0.1 ... linking ... done.
Loading package pcre-light-0.4 ... linking ... done.
Loading package containers-0.5.0.0 ... linking ... done.
Loading package attoparsec-0.10.4.0 ... linking ... done.
Loading package blaze-textual-0.2.0.8 ... linking ... done.
Loading package base16-bytestring-0.1.1.6 ... linking ... done.
Loading package mysql-0.1.1.7 ... linking ... done.
Loading package mysql-simple-0.2.2.4 ... linking ... done.
```

You can check with the *mysql* command line utility for the created table:

```
mysql> use test;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
Database changed
mysql> desc users;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id | int(11) | YES | | NULL | |
| fname | varchar(80) | YES | | NULL | |
| lname | varchar(80) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
```

```
+-----+-----+-----+-----+-----+-----+
| id | int(11) | YES | | NULL | |
| fname | varchar(80) | YES | | NULL | |
| lname | varchar(80) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

You can now list the databases available in the MySQL server using the *query_* function as illustrated below:

```
{-# LANGUAGE OverloadedStrings #-}
import Database.MySQL.Simple
main :: IO ()
main = do
    conn <- connect defaultConnectInfo
    { connectUser = "user"
    , connectPassword = "password"
    , connectDatabase = "test"
    }
    databases <- query_conn "SHOW databases"
    print (databases :: [Only String])
    close conn
```

You can compile the above code directly with GHCi and execute the main function, as follows:

```
$ ghci show-databases.hs
GHCi, version 7.6.3: http://www.haskell.org/ghc/ ? for help
Loading package ghc-prim ... linking ... done.
Loading package integer-gmp ... linking ... done.
Loading package base ... linking ... done.
[1 of 1] Compiling Main ( show-databases.hs, interpreted )
Ok, modules loaded: Main.
ghci> main
Loading package array-0.4.0.1 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
[Only {fromOnly = "information_schema"}, Only {fromOnly = "mysql"}, Only {fromOnly = "performance_schema"}, Only {fromOnly = "test"}]
```

You can try inserting a record into the database using the *execute* function:

```
execute conn "insert into users (id, fname, lname) values (?, ?, ?)" ["1" :: String, "Edwin" :: String, "Brady" :: String]
```

After running the code, you can check the database entry using the *mysql* client program as shown below:

```
mysql> select * from users;
+-----+-----+-----+
| id | fname | lname |
+-----+-----+-----+
```